

题解合集

生成时间: 2026/6/26 23:26:26 题目数量: 8

成功: 8 个 | 跳过(无题解): 0 个

题目 1: 方块消除

题目链接: /d/CSPJplan/p/1

题解

方块消除

题目大意

有一堆由红色（0）和蓝色（1）方块组成的垂直堆叠塔，允许反复消除任意一对相邻且颜色不同的方块（红+蓝或蓝+红）。消除后上方方块下落，问最多可以移除多少个方块。

直观思路

每次只能消除一对相邻异色方块，这有点像对字符串做“相邻异色消除”操作。最终能消除的方块数正好是能消除的异色对子数的两倍。

如果把这题看成“消除所有相邻异色方块对，直到无法继续”，其实就是模拟消除异色相邻方块的过程。

该过程等价于经典的“配对消除”思路：用栈维护方块，遍历序列，如果当前方块与栈顶颜色不同，就消除栈顶的方块（pop），累积计数；否则入栈。

最后，计数乘以2即为最大消除数量。

该算法无需模拟方块下落，因为每次操作消除的是相邻的异色对，且下落即相当于压缩字符串。

关键代码

```
// 用栈维护未被消除的方块颜色
// 遍历字符串，遇到与栈顶颜色不同的就消除一对
int maxRemovePairs(const string &S) {
    vector<char> stk; // 模拟栈存放0/1
    int count = 0;    // 记录消除的对数
    for (char c : S) {
        if (!stk.empty() && stk.back() != c) {
            stk.pop_back();
            count++;
        } else {
            stk.push_back(c);
        }
    }
    return count * 2; // 每对两块
}
```

题目反思

- 关键观察**：消除后下落等价于压缩字符串，实现自动“相邻异色配对”消除。用栈维护连续待匹配元素是简洁做法。
- 易错点**：不要试图复杂模拟下落过程，只需栈配对。
- 时间复杂度**： $O(N)$ ，只遍历一次字符串，栈操作均摊 $O(1)$ 。
- 思维技巧**：此题是类似“去除相邻反向字符”、“配对消除”问题的典型，栈匹配思想十分有效。

这种题目同时训练对栈以及贪心/字符串消除的理解，是字符串和数据结构结合的好题。

语言: C++

模型: claude-opus-4-7

生成时间: 2026-06-26T12:10:10.057Z

题目 2: 划分问题

题目链接: /d/CSPJplan/p/2

题解

```
# 划分问题

## 题目大意
给定 $N$ 个整数难度值，$N$ 为偶数。将难度划分为两组：难度不小于 $K$ 为 ARC，用问题，难度小于 $K$ 为 ABC 用问题。求使得两组数量相等的最小 $K$。

## 直观思路
- 题中要求 ARC 和 ABC 各有 $\frac{N}{2}$ 个问题。
- 排序后，ABC 用问题是难度较小的 $\frac{N}{2}$ 个，ARC 用问题是难度较大的 $\frac{N}{2}$ 个。
- 给定 $N$ 是偶数，排序后中间位置的元素索引为 $N/2$ 和 $N/2+1$ (假设从 $1$ 开始编号)。
- $K$ 需要满足：$K$ 大于等于第 $N/2+1$ 个元素难度，又严格大于第 $N/2$ 个元素难度。
- 因为 $K$ 是整数，所有满足
  $$
  d_{\frac{N}{2}} < K \leq d_{\frac{N}{2}+1}
  $$
  的整数 $K$ 都满足划分条件。

## 关键代码
```C++
// d 已排序，n 为问题数且为偶数
int count = d[n/2] - d[n/2 - 1]; // 下标从0算，第n/2-1和n/2对应题目中第N/2和N/2+1个元素
// count 表示满足条件的整数 K 的数量，可能为0
```

## 题目反思

- 易错点**：数组下标要注意是 0 基还是 1 基。\$K\$ 是整数且区间是开闭的组合（左开右闭），差值小于等于0则无解。
- 关键观察**：将数组排序后，\$N/2\$ 分割点的左右两边正好给出 ABC 和 ARC 队列的界限。
- 本质是查找中间两个分界点之间的整数个数。

```
语言: C++

模型: claude-opus-4-7

生成时间: 2026-06-26T12:10:33.177Z

题目 3: 学生入校顺序

题目链接: [/d/CSPJplan/p/3] (/d/CSPJplan/p/3)

题解

学生入校顺序

题目大意

已知班级中 N 个学生的学号为 1 到 N，每个学生到校时教室中已有（包括自己）A_i 个人。A_i 是学生 i 到校时的“到校人数序号”。

输入是数组 A，A_i 表示学号 i 的学生是第 A_i 位到校的学生。要求输出按到校顺序排列的所有学号。

直观思路

- A_i 是学号为 i 的学生的到校顺序编号（即第 A_i 个到校）。
- 题目实际上是将每个学号 i 放到位置 A_i。
- 根据数组 A，直接构造一个数组 `ans`，`ans[A[i]] = i` 即可。

关键代码

```C++
// 将学号 i 放到第 A[i] 位
for (int i = 1; i <= N; i++) {
```

```
    ans[A[i]] = i;
}
```

题目反思

- 易错点**：注意题目中 A_i 与学号 i 的关系，是到校顺序对应学号，而不是学号对应位置。
- 关键观察**： A 数组就是学生到校顺序的映射，反向解码即可。
- 效率**：时间复杂度为 $O(N)$ ，空间复杂度为 $O(N)$ ，满足大数据量需求。

注：该题强调映射理解，属于简单模拟题。只要理解 A_i 表示“第 A_i 个到校的是学号为 i ”即可直接建立映射。

语言: C++

模型: claude-opus-4-7

生成时间: 2026-06-26T12:10:45.391Z

题目 4: 最小矩形面积

题目链接: [/d/CSPJplan/p/4](#)

题解

最小矩形面积

题目大意

给定平面上 N 个点，要求找到一个边平行坐标轴的矩形，该矩形内（包含边界）至少包含 K 个点，求所有满足条件的矩形中面积的最小值。

直观思路

- 矩形的左、右边界一定是由点的横坐标确定，底、顶边界由点的纵坐标确定。
- 因为点的坐标互不相同，可以先将所有点的 x 和 y 坐标分别排序。
- 枚举所有可能的边界组合（即枚举 x 的左边界和右边界，枚举 y 的下边界和上边界），计算包含的点数是否达到 K 。
- 找出符合条件的最小面积。

由于 $N \leq 50$ ，四重循环枚举边界耗时 $O(N^4)$ 复杂度可接受。

关键代码

```
// a, b 分别存储所有点横、纵坐标排序后的数组
// x[], y[] 存储原始点坐标

int ans = LLONG_MAX;
for(int i = 1; i <= n; i++) {
    for(int j = i; j <= n; j++) { // 左右边界, j>=i保证非负宽度
        for(int k = 1; k <= n; k++) {
            for(int l = k; l <= n; l++) { // 上下边界, l>=k保证非负高度
                int cnt = 0;
                for(int u = 1; u <= n; u++) {
                    if(x[u] >= a[i] && x[u] <= a[j] && y[u] >= b[k] && y[u] <= b[l]) cnt++;
                }
                if(cnt >= d) {
                    ans = min(ans, (a[j] - a[i]) * (b[l] - b[k]));
                }
            }
        }
    }
}
```

关键实现细节

- 对输入点坐标横纵分别排序，方便枚举边界。
- 留心 $j \geq i$ 和 $i \geq k$ ，保证面积非负。
- 面积乘积可能非常大，需使用 64 位整数（`long long`）。
- 包含边界条件用 `>=` 和 `<=` 进行判断。
- 最后输出最小面积。

复杂度分析

- 时间复杂度： $O(N^4)$ ，四重循环枚举矩形边界， $N=50$ 时勉强可行。
- 空间复杂度： $O(N)$ ，存储坐标数组。

题目反思

- 关键观察**：矩形边界必定选择点坐标作为边界，减少搜索空间。
- 易错点**：需要包含边界的点，乘法溢出需用 64 位整型，注意左/右、上/下边界的枚举顺序。
- 思维技巧**：离散化坐标并枚举边界，是这类包含至少 K 个点的最小矩形问题的常用策略。
- 类似题目**：AtCoder ABC 075 D 有相似题解思路，枚举点坐标作为矩形边界。

语言: C++

模型: claude-opus-4-7

生成时间: 2026-06-26T12:11:10.012Z

题目 5: 聚集的孩子们

题目链接: /d/CSPJplan/p/5

题解

题解：聚集的孩子们

题目大意

给定一个由字符 `R` 和 `L` 组成的字符串 S （首字符为 `R`，末字符为 `L`），在每个格子初始站一个孩子。孩子们根据所在格子的字符，一次移动中向右（`R`）或向左（`L`）移动一格。经过非常多次（ 10^{100} 次）移动后，问每个格子上有多少孩子。

直观思路

- 由于移动次数极大，且孩子位置只受左右相邻两个格点影响，状态将趋于稳定且周期性。
- 关键位置是字符串中的“**RL交界处**”：即某个位置 r ， $S_r = R$ ， $S_{r+1} = L$ 。
- 孩子只会在这两个交界处左右两个格子上最终停留，因为经过很多步后，孩子在 `R` 方向一直向右走，`L` 方向一直向左走，到达 RL 边界后会停留交界处的两个格子之间往返。
- 本题的核心是统计在每个这样的边界处，孩子们最终在这两个位置的分布数量。

核心规律

考虑一段 `RR...RLL...L`，中间转折点是 $(r, r+1)$ 。将这段连续的 R 串和 L 串长度分别记为 cur 。

- 这 cur 个孩子会汇聚在位置 r 和 $r+1$ 两个格子上。
- 具体分布规则：
 - 在第 r 个格子上的孩子数是 $(cur + 1) / 2$ （向上取整）
 - 在第 $r+1$ 个格子上的孩子数是 $cur / 2$ （向下取整）

这个划分是基于“经过足够多次移动，孩子们会在这两个位置轮流交换，最终孩子数平均分配到两个位置上，奇偶位置形成的分布不一样”。

算法步骤

- 遍历字符串，统计连续相同字符的长度 `cur`。

2. 当遇到 **RL** 边界时，将这段连续字符数 **cur** 分配到边界两侧格子：
- $p[r] += (cur+1)/2$
 - $p[r+1] += cur/2$
3. 当遇到 **LR** 边界时（因为最左是 R，最右是 L，这种情况出现时需要反过来分配）：
- $p[r] += cur/2$
 - $p[r+1] += (cur+1)/2$
4. 结束时，将最后一段剩余计数分配。
5. 输出最终每个位置的孩子数。

关键代码段

```
// a 为输入字符数组, p 为结果数组
int r = 1; // 记录最近一个RL边界位置
int cur = 1; // 当前连续相同字符段长度

for(int i=2; i<=n; i++){
    if(a[i] == a[i-1]){
        cur++;
    }
    else if(a[i-1] == 'R'){ // RL 边界
        r = i-1;
        p[r] += (cur + 1) / 2;
        p[r + 1] += cur / 2;
        cur = 1;
    }
    else { // LR 边界
        p[r] += cur / 2;
        p[r + 1] += (cur + 1) / 2;
        cur = 1;
    }
}
// 最后一段处理
p[r] += cur / 2;
p[r + 1] += (cur + 1) / 2;
```

复杂度分析

- 时间复杂度 $O(N)$ ，一次线性扫描字符串完成。
- 空间复杂度 $O(N)$ ，存储结果数组。

题目反思

- 关键观察**：孩子最终聚集在 RL 边界处两格子上的规律，以及如何将连续字符段长度正确分配到两个位置。
- 易错点**：区分 RL 和 LR 两种边界的处理方式，特别是起点和终点的边界段。
- 思维技巧**：将复杂的多次移动归约为稳定周期状态，利用数学上的划分和取整技巧，结合初始条件和边界。
- 本题类似于“跳跃和循环轨迹”题，可以归纳孩子的最终落点与字符分段对应关系。

此题考察字符串的分段统计，状态稳定性判断与数学分配的能力。只要掌握 RL 边界处孩子分布规则，即可高效解决。

语言: C++

模型: claude-opus-4-7

生成时间: 2026-06-26T12:11:12.924Z

题目 6: 移动棋子

题目链接: [/d/CSPJplan/p/6](#)

题解

移动棋子 题解

题目大意

给定一个长度为 N 的排列 P 和对应棋盘每个格子的分值 C ，棋子初始放在某个格子（得分0），每次移动时棋子从当前格子 i 跳到 P_i 对应的格子，得分增加 C_{P_i} 。

总共可移动 K 次（ $1 \leq K \leq 10^9$ ），求最终可获得的最大得分。

关键问题和难点

- N 最多 5000，允许 $O(N^2)$ 复杂度。
- K 最大可达 10^9 ，无法模拟所有移动。
- 给定排列形成多个“环”，移动即在这些环上跳转。
- 需要针对是否循环和循环得分总和正负进行不同处理。
- 目标是寻找起点和移动次数的最佳策略，实现最大和。

算法思路

1. 环的分解与分析

- 由于 P 是排列，所有位置分成若干不相交的环。
- 对于每个位置 u ，从 u 出发，顺着 P 不断跳转，直到回到 u ，得到一个环。
- 按环周期记为 t ，环内分值列表为 $a = [c_{p_1}, c_{p_2}, \dots, c_{p_t}]$ 。

2. 环内得分累积与部分步数求最优

- 对环内序列构造前缀和数组 s : $s_0=0, s_i = \sum_{j=0}^{i-1} a_j$
- 对于 $k \leq t$ ，计算环内连续移动 k 步的最大得分就是求 $\max_{1 \leq i \leq k} s_i$ 。

3. 长距离移动考虑圈和的正负

- 设环的总和为 $\text{sum} = \sum_{i=0}^{t-1} a_i$
- 设 $r = \lfloor K / t \rfloor$ 为整圈次数， $q = K \bmod t$ 为剩余步数。
- 情况 A : $\text{sum} < 0$** ，整圈加分为负
 - 不宜多走整圈，最多走一次部分环。
 - 最大价值即为环内连续步数不超过 K 内最大连续和。
- 情况 B : $\text{sum} \geq 0$** ，整圈加分为非负
 - 可以考虑多走完整的环圈。
 - 利用 r 次整圈加分，剩余步数走环内的前 q 步。
 - 计算最大得分时要同时考虑：
 - 只走部分步数获得最大连续和（不足一圈）
 - 走若干圈后加上剩余的部分步数

4. 对所有起点尝试，求最大值

- 对每个起点所在环：
 - 枚举环内 t 个起点，计算最大连续前缀和。
 - 利用上述策略计算最大分数
- 取全局最大。

关键代码片段

```
// 对每个位置 u 构造环
for(int u=1; u<=n; u++) {
    vector<int> a;
    int d = u;
    int sum = 0;
    do {
        d = p[d];
        a.push_back(c[d]);
        sum += c[d];
    } while(d != u);

    int t = a.size();
    int r = k / t, q = k % t;
```

```
// 前缀和数组
vector<long long> s(t+1, 0);
long long maxn = LLONG_MIN;
// 环内前缀最大和
for(int i=1; i<=t; i++){
    s[i] = s[i-1] + a[i-1];
    if(s[i] > maxn) maxn = s[i];
}

// q 部分步数最大和
long long max2 = LLONG_MIN;
for(int i=1; i<=q; i++) {
    if(s[i] > max2) max2 = s[i];
}

if(q < t) {
    if(sum < 0) {
        ans = max(ans, maxn);
    } else if(r == 0) {
        ans = max(ans, max2);
    } else {
        ans = max(ans, max({maxn + sum * max(r-1, 0LL), max2 + sum * r, sum * r}));
    }
} else {
    if(sum < 0) {
        ans = max(ans, maxn);
    } else {
        ans = max(ans, sum * r + maxn);
    }
}
}
```

复杂度分析

- 枚举起点 N 次，每次找到环的大小最多是 N ，整体环遍历每个元素不超过 N 。
- 每个环计算前缀和、最大和是 $O(t)$ ，总共 $O(N)$ 。
- 时间复杂度： $O(N^2)$ 适合 $N=5000$ 。
- 空间复杂度： $O(N)$ 用于存储数据。

题目反思

- 关键在于将整体移动理解为若干环中的跳跃，避免大 K 带来的爆炸。
- 正负循环和的处理是难点，避免盲目多跑圈造成分数减小。
- 利用前缀和找最大连续和是关键技巧，抓住部分环路求解。
- 充分利用循环结构和环长度分解问题。
- 这种题目在处理大步数路径得分时很常见，比如比赛中滑动区间最大值、环状DP优化。

这就是本题的系统分析和核心算法思路。希望能让你清晰理解环路结构下大步数移动的动态规划求解方案。祝你竞赛顺利！

语言: C++

模型: claude-opus-4-7

生成时间: 2026-06-26T12:11:19.772Z

题目 7: RGB三元组计数

题目链接: </d/CSPJplan/p/7>

题解

RGB三元组计数 题解

题目大意

给定长度为 N 的字符串 S （只包含 'R', 'G', 'B'），找出满足以下条件的三元组 (i, j, k) 个数：

- $\forall i < j < k \leq N$
- 三个字符两两不同，即 $S_i \neq S_j \neq S_k \neq S_i$
- $j - i \neq k - j$ （即三点不等距）

直观思路

- 枚举 i, j ，因为三元组有序且 $i < j < k$ 。
- 确定 S_i 与 S_j 不同后，利用字符总共有三种，得到第三个字符 c 。
- 统计 k 中满足 $k > j$ 、 $S_k = c$ 且 $k \neq 2j - i$ （防止等距三元组）有多少个。
- 维护三个颜色的前缀和数组，可以在 $O(1)$ 时间内查询 k 位置之后某颜色的数量。

关键实现细节

- 将字符转换成数字表示，例如：R=1, G=2, B=3；
- 使用三组前缀和数组 $s[1\dots 3][1\dots N]$ ，其中 $s[c][i]$ 表示前 i 个字符中颜色为 c 的数量；
- 对每组 (i, j) ，先判断颜色是否不同，再计算第三个颜色 $t = 6 - a[i] - a[j]$ （因为 $1+2+3=6$ ）；
- 计算所有满足条件的 k 数量为 $s[t][N] - s[t][j]$ ，再排除 $k = 2j - i$ 这一位置是否是颜色 t ，相当于剔除等距三元组；
- 累加计数。

代码核心片段

```
// 预处理前缀和
for(int i = 1; i <= n; i++) {
    for(int c = 1; c <= 3; c++) {
        s[c][i] = s[c][i-1];
    }
    s[a[i]][i]++;
}

long long ans = 0;
for(int i = 1; i <= n; i++) {
    for(int j = i + 1; j <= n; j++) {
        if(a[i] != a[j]) {
            int t = 6 - a[i] - a[j]; // 第三种颜色
            // 计算满足 k > j 且 S_k = t 的数量
            int cnt = s[t][n] - s[t][j];
            int k = 2*j - i; // 等距三点位置
            if (k <= n && a[k] == t) {
                cnt--; // 排除等距三元组
            }
            ans += cnt;
        }
    }
}
```

复杂度分析

- 时间复杂度： $O(N^2)$ ，双重循环枚举 (i, j) ，对于每对执行 $O(1)$ 查询。
- 空间复杂度： $O(N)$ ，前缀数组存储颜色计数。

由于 $N \leq 4000$ ， $O(N^2)$ 方案可接受。

题目反思

- **关键观察**：三种颜色的总和是 6 ，可以用 $6 - a[i] - a[j]$ 快速求得第三个颜色，简明高效。
- **去除等距三元组**：利用 $k = 2j - i$ 判断是否构成等距三元组，并剔除，避免重复计数。
- **易错点**：
 - 越界判断 $k = 2j - i$ 是否超过字符串长度。
 - 三个字符必须两两不同，不能只比较部分。
- **技巧提示**：
 - 前缀和数组的使用极大地减少了查询次数，提升效率。
 - 颜色使用数字映射更方便计算，避免字符串比较。

这道题有点类似“三色分类统计+等距条件排除”的组合计数，灵活运用前缀和和数学技巧即可高效解决。

如果你想，我还能帮你写更详细的代码讲解或进一步的优化策略！

语言: C++

模型: claude-opus-4-7

生成时间: 2026-06-26T12:11:18.976Z

题目 8: 轰炸机

题目链接: /d/CSPJplan/p/8

题解

轰炸机 — 题解

题目大意

给定一个 $H \times W$ 的网格和 M 个爆破目标的位置，炸弹放置在某个格子时可以摧毁该位置所在行和列上的所有目标。求能摧毁的最多目标数量。

直观思路

- 计算每一行、每一列的爆破目标数量。
- 找出行中目标数最多的行集合，列中目标数最多的列集合。
- 试图找到一个交点（行、列）该位置上没有目标，表示炸弹放这里可以摧毁的目标数是行最大值 + 列最大值；
- 如果所有最大行和最大列交点位置都有目标，说明交点必须有目标，答案比行最大值 + 列最大值少1。

详细分析

- 统计每行和每列的目标数**
使用数组 `rCount` 统计行上目标数，数组 `cCount` 统计列上目标数。
- 找最大行数和最大列数**
例如最大行目标数为 `maxR`，最大列目标数为 `maxC`。
- 找出所有达到最大行目标数的行集合** `maxRRows`
找出所有达到最大列目标数的列集合 `maxCCols`。
- 判断交点是否存在空格**
遍历所有 `(r, c)` 组合：
 - 如果该位置上没有目标，则炸弹放这里达到的最大摧毁数为 `maxR + maxC`。
 - 否则，如果所有坐标都有目标，答案是 `maxR + maxC - 1`。

关键代码片段

```
// rCount[i], cCount[i] 分别统计第 i 行和第 i 列目标数
int maxR = 0, maxC = 0;
vector<int> maxRRows, maxCCols;

for (int i = 1; i <= H; i++) {
    if (rCount[i] > maxR) {
        maxR = rCount[i];
        maxRRows.clear();
        maxRRows.push_back(i);
    } else if (rCount[i] == maxR) {
        maxRRows.push_back(i);
    }
}

for (int i = 1; i <= W; i++) {
    if (cCount[i] > maxC) {
        maxC = cCount[i];
        maxCCols.clear();
        maxCCols.push_back(i);
    }
}
```

```
    } else if (cCount[i] == maxC) {
        maxCCols.push_back(i);
    }
}

// 判断是否存在无爆破目标的交点
bool found = false;
for (auto &r : maxRRows) {
    for (auto &c : maxCCols) {
        if (mp.find({r, c}) == mp.end()) {
            found = true;
            break;
        }
    }
    if (found) break;
}

int ans = maxR + maxC - (found ? 0 : 1);
cout << ans << "\n";
```

复杂度分析

- 时间复杂度：
 - 统计行列爆破目标数 $O(M)$
 - 查找最大行列 $O(H+W)$
 - 遍历可能的最大行列交点 $O(\text{maxRRows.size} * \text{maxCCols.size}) \leq O(M)$
- 空间复杂度：
 - 需要存储行、列统计数组，大小 $O(H+W)$
 - 存储目标位置哈希， $O(M)$

题目反思

- 关键观察：**

最大的摧毁目标数 = 行最大值 + 列最大值 或 行最大值 + 列最大值 - 1，关键在于是否存在空交点。
- 易错点：**
 - 如果最大行列交点全是目标，答案会错误少算1。
 - 行数和列数很大，不能直接用二维数组存储目标位置，哈希或 `map` 必须。
- 思维技巧：**

利用“极值点”的交叉判断，巧妙避免暴力遍历所有网格。
- 类似题目：**

与“二元组最优覆盖问题”“矩阵投影最大值”等问题思路相似。

该题充分考察了对二维数据统计和空间优化的能力，以及巧妙的极值点思考，是一个典型的冲刺卡点题。

语言: C++

模型: claude-opus-4-7

生成时间: 2026-06-26T12:11:22.259Z